Recall-Bounded Durability for Graph-Based Approximate Nearest Neighbor Indexes

Anonymous
`Under submission`

July 14, 2026

Preliminary single-host study — work in progress

Abstract

Approximate nearest neighbor (ANN) search is evaluated almost entirely on two axes: recall and queries per second. Durability—whether an acknowledged write survives a crash or power loss—is largely absent from the ANN research literature and from the mainstream ANN benchmarks, and is unquantified even in systems that provide it. This paper makes durability a first-class axis for graph ANN. Its foundation is a reusable protocol for measuring durability honestly and a characterization of its cost on one host; its headline is *recall-bounded durability*, a guarantee that expresses and bounds the damage a crash can do in recall units—the one primitive here that is new (the storage mechanisms we apply, group commit and snapshot recovery, are textbook). Using a from-scratch C++20 vector database with a segmented, write-ahead-logged (WAL) HNSW store—whose index reaches 0.999 recall@10 on SIFT1M (and 0.98 on the harder 960-d GIST1M) and sits on the hnswlib recall–QPS Pareto front, confirming we measure on a competitive, not a broken, index—we contribute the following in a preliminary single-host study (Apple M4 Pro). (1) A *durability-aware measurement protocol* for ANN indexes: a tmpfs (RAM-backed filesystem) guard, an `fsync`-floor baseline, and mandatory `fsync`-mode labeling, because on some platforms the default `fsync` does not flush the drive cache and so reports optimistic "durable" numbers. (2) An illustration of that protocol catching an `fsync`-mode trap: on macOS/APFS, where plain `fsync` is a page-cache no-op, switching to honest per-write durability (`F_FULLFSYNC`) lowers the device's durable-sync ceiling by ∼155× (49,199 to 317 syncs/s) in a tight-loop microbenchmark. We foreground this as a cautionary example of how mislabeled numbers arise, not as a headline system result; it is a macOS/APFS artifact. (3) An illustrative durability-tax and group-commit characterization on the SIFT-scale (d=128) index: honest per-write durability cuts insert throughput ∼4.5× (442 to 98 inserts/s), and a group-commit *batch-size sweep* then recovers the tax along a curve—from 1× at batch 1 to 4.3× at batch 2,000 under `F_FULLFSYNC`, while under plain `fsync` batching barely helps (∼1×), showing the speedup is a function of both batch size and `fsync` mode rather than a single constant—and a fork-and-crash test (child `_exit`s without clean shutdown) checks crash-consistency. (4) An illustrative recovery comparison: snapshot-load of a sealed segment is near-constant while WAL replay grows with N (a 5× to 32× gap over N=1k–6k); we report this as motivation for periodic sealing, noting it omits sealing/snapshot-creation cost, and position it against P-HNSW's persistent-memory recovery analysis. (5) *Recall-bounded durability*: we express the exposure of group commit's un-synced window in recall units and bound it (worst case $\min(W, k)/k$; benign expectation W/N), measuring on SIFT1M that a crash risks only 0.13% recall@10 at our batch size—about ∼770× below the distribution-free worst case—so a recall-staleness SLA and group-commit throughput coexist. All code, harnesses, and results are open and reproducible; every durability number is the *median of 7 trials* with the observed min–max range reported alongside.

1 Introduction

Graph-based ANN indexes—Hierarchical Navigable Small World (HNSW) graphs [1] and their descendants—dominate in-memory vector search, and vector databases built on them now back retrieval-augmented generation and semantic search at scale. The community evaluates them on *recall@k* (the fraction of a query's true k nearest neighbors the index returns) and *throughput* (queries/s, QPS), summarized by the recall-vs-QPS Pareto frontier popularized by ANN-Benchmarks [13] and the Big ANN challenges [14]. What these evaluations almost never measure is *durability*: does an acknowledged write survive a crash?

This is not a niche concern. Vector databases are increasingly operational stores that ingest and update continuously, and an operational store that loses acknowledged writes is not a database. Yet mainstream ANN libraries exclude durable storage entirely (Faiss offers manual serialization, no WAL [10]; hnswlib offers soft-delete tombstones and no crash consistency [2]), and none of the widely used ANN benchmarks (ANN-Benchmarks [13], Big ANN [14], VectorDBBench [15]) include a crash-recovery, `fsync`, or durability case. Production vector databases (Milvus [12], Qdrant, Weaviate) do implement WAL-based persistence, but publish no isolated `fsync`-tax, crash-consistency, or recovery-time analysis. The closest research systems either integrate ANN into an operational DB and *inherit* durability from the host engine—delegating rather than analyzing it [9]—or, like LSM-VEC [8], build an LSM-tree graph index yet report no `fsync` cost, crash consistency, or recovery time.

Our stance is deliberately measurement-oriented: build durability into a *standalone* graph-ANN index using well-known techniques, then measure it honestly and report where the naive measurement misleads. We do not claim the *storage* mechanisms as novel—group commit and snapshot-based recovery are textbook database techniques—and the main contribution is the measurement protocol and the (single-host, median-of-7-trial) characterization it produces. The one genuinely new mechanism is conceptual and ANN-specific: expressing and bounding a crash's damage in *recall* units (§6), which only makes sense because the index is approximate. Two observations motivate the work. First, durable writes are expensive in a way that ANN evaluation has never quantified, and the expense is easy to hide: on macOS (and any stack where `fsync` does not force a drive-cache flush) the "durable" number is a page-cache number. Second, once that cost is paid, the standard techniques apply, but their effect on an ANN index (where each insert also mutates a proximity graph) has not been reported.

Contributions.

- **C1 (§3).** A durability-aware measurement protocol for ANN: tmpfs detection, an `fsync`-floor baseline, and `fsync`-mode labeling, so ANN write/durability numbers are comparable and honest. This is the paper's primary, reusable contribution.
- **C2 (§4).** An illustrative ANN-isolated durability-tax measurement—throughput and latency percentiles vs. the strength of the durability guarantee—applying the protocol on one host, presented as an example of what it surfaces rather than a general result.
- **C3 (§5).** A group-commit *batch-size sweep* with a fork-and-crash consistency check, showing the recovered throughput is a curve in batch size and depends on `fsync` mode—not a single headline speedup.
- **C4 (§7).** A recovery comparison (snapshot-load vs. WAL-replay) for a standalone graph ANN on conventional block storage (`fsync`/WAL)—extending P-HNSW's [11] persistent-

memory recovery analysis to this setting—reported with its caveats (snapshot-creation cost excluded).

- **C5 (§6).** *Recall-bounded durability*: to our knowledge the first durability guarantee for an ANN index stated in *recall* units. We bound the recall a crash can destroy—worst-case $\min(W, k)/k$ and benign expectation W/N for an un-durable window of W inserts—and measure that on SIFT1M the real exposure is $\sim$770$\times$ below the worst case (0.13% at our batch size), so group commit's throughput and a recall-staleness SLA are compatible.

2 Background and System

Our testbed is a from-scratch C++20 vector database. The default storage engine is a Qdrant/LSM-style *segmented store*: a single mutable segment backed by a write-ahead log (WAL), plus N immutable *sealed* segments, each carrying an HNSW snapshot. Writes append to the WAL; when a size or tombstone-ratio threshold is crossed, the mutable segment is sealed (its HNSW graph and vectors serialized), and sealed segments are compacted in the background. A cross-segment map resolves the current location of each key, versioned by sequence number (multi-version concurrency control, MVCC).

Durability model. Every WAL record is framed with a header (magic, version, op, sequence, field lengths, dimensions) and a CRC-32 covering the header and payload, and—by default—is `fsync`'d before the call returns. Snapshot and manifest writes are published atomically (temp file $\rightarrow$ `fsync` $\rightarrow$ rename $\rightarrow$ directory `fsync`). Recovery of a sealed segment loads its HNSW snapshot directly (memcpy of neighbor lists and precomputed distances) rather than replaying the WAL through the graph builder; the mutable segment replays its (small) WAL. WAL replay stops at the first record failing the magic/version/CRC check, tolerating a torn tail.

Index. Search uses HNSW with the Malkov–Yashunin diversifying neighbor-selection heuristic [1]. The search-time candidate-list width ef (and its build-time analogue $\text{ef}_{\text{construction}}$) trade recall for speed. Distance kernels are SIMD-vectorized (NEON on our host; an AVX+FMA path is implemented but untested here) with a devirtualized hot path. After we fixed an initially naïve neighbor-selection rule—under which recall plateaued regardless of ef—recall@10 on SIFT1M rises monotonically with ef to 0.999 and sits on the same recall–QPS Pareto front as hnswlib (Table 5); i.e. the index is competitive before we layer durability on top.

3 Measuring Durability Honestly

Durability numbers are worthless without a statement of *what* was made durable. The pitfalls below are well known in the storage-systems literature— application-level crash-consistency is hard to get right [17], commodity stores mishandle power faults [18], and even PostgreSQL mishandled `fsync` error reporting [19]. Our contribution is not to discover them but to port this discipline to ANN evaluation, where it is absent. Two traps recur.

The tmpfs trap. Benchmarks frequently run under `/tmp`, which on some Linux distributions is a RAM-backed `tmpfs` where `fsync` is a no-op. A "durable" insert benchmark on tmpfs measures the CPU path only. Our harness `statfs`-checks the target and *refuses* to report write numbers on tmpfs.

Table 1: `fsync` floor on the testbed (Apple M4 Pro, APFS/NVMe): durable syncs/s in a tight `write()+sync` loop, median of 7 trials (min–max). Plain `fsync` does not flush the drive cache; `F_FULLFSYNC` does. Device microbenchmark; the ∼155× gap is a macOS/APFS artifact, not a property of the index.

Mode	Durable syncs/s (median, min–max)
Plain `fsync` (page cache)	49,199 (41,983–51,782)
`F_FULLFSYNC` (drive-cache flush)	317 (291–324)

Table 2: Durability tax: single-insert throughput and latency vs. `fsync` mode (Apple M4 Pro, d=128, per-write durability, N=2,000 inserts/trial), median of 7 trials (min–max). Honest durability cuts throughput to ∼22% (4.5×) and roughly triples the p99 (99th-percentile) tail.

Mode	Inserts/s	p50 (µs)	p99 (µs)
Plain `fsync`	442 (408–476)	1,732 (1,435–1,896)	14,425 (14,114–14,915)
`F_FULLFSYNC`	98 (98–106)	8,097 (7,972–8,226)	48,372 (35,252–49,390)

The `fsync`-mode trap. On macOS, `fsync`(2) does not flush the drive's write cache; true power-loss durability requires `fcntl(F_FULLFSYNC)`, which is far slower. A number labeled "durable" under plain `fsync` on macOS overstates durability. We therefore (a) expose an `F_FULLFSYNC` toggle and (b) report an *`fsync` floor*: a tight `write()+sync` loop giving the device's durable-sync ceiling, so index overhead can be separated from the device's flush cost. Table 1 shows the floor differs by ∼155× between the two modes on our testbed. This large gap is the *illustration* of why `fsync`-mode labeling matters, not a headline system result: it is a microbenchmark of the device (not of our index) and a macOS/APFS-specific artifact that will be far smaller on Linux/NVMe. The transferable point is the trap—a number labeled "durable" under plain `fsync` on this platform is a page-cache number—not the specific 155×.

Methodology and statistics. Every durability number in this paper is the *median of 7 trials* on one host, taken after a warm-up pass, and we report the observed min–max range alongside each median so the reader can see the run-to-run spread. We use median-and-range rather than a parametric confidence interval because 7 trials is too few to justify a distributional assumption; the ranges are wide enough in places (e.g. the plain-`fsync` floor spans −15% to +5% around its median) that we read differences smaller than the ranges as noise. Generalizing across media and hosts remains future work (§10). The SIFT1M recall/QPS numbers (§8) are a single deterministic sweep, as recall is not stochastic at fixed parameters.

4 The Durability Tax

We define the *durability tax* as the throughput an index loses and the latency it adds to guarantee that acknowledged writes survive. We measure it by inserting one record at a time (each `fsync`'d before returning) into the segmented store and varying *only* the `fsync` mode, on the SIFT-scale configuration (d=128, a real HNSW insertion with $\text{ef}_{\text{construction}}$=200). Because the two rows in Table 2 run the identical index work and differ only in whether the per-insert flush actually reaches the drive, the difference between them isolates the durability cost.

Three readouts. First, honest durability is not free: throughput falls to about a fifth and the p99

tail roughly triples, because each insert now waits on a real drive flush. Second, we do *not* attempt to decompose the absolute per-insert latency by subtracting the single-sync `fsync` floor: the insert path performs more than one durable operation per record (a WAL record flush plus, on segment growth, a metadata/directory flush), so under `F_FULLFSYNC` each such operation pays a full drive flush while under plain `fsync` they are page-cache-cheap. The floor is therefore a per-*single*-sync lower bound on the flush cost, not a clean subtrahend—which is exactly why the honest-vs-plain *ratio*, not a floor subtraction, is the right way to read the tax. Third—and this is why the `fsync` floor matters—under plain `fsync` the bottleneck is the graph insertion, not the sync, so a system that stops at plain `fsync` both understates its true durability cost *and* understates how much a batching optimization can help. That is the subject of §5.

Scale caveat. These tax and group-commit numbers are at N=2,000 inserts, whereas the competitiveness and recall-bounded results are at N=10^6. Per-insert graph work grows $\sim\log N$, so at production scale the graph term grows relative to the fixed flush cost: we expect the 4.5× tax ratio to *shrink* and the plain-vs-honest convergence to arrive at smaller batches. Confirming this with an N-sweep to 10^6 is future work (§10).

Dimension robustness check. Repeating the measurement at d=64 gives near-identical numbers—`fsync` floor 48,216 vs. 49,199 (plain) and 323 vs. 317 (`F_FULLFSYNC`); per-write 413 vs. 442 (plain) and 102 vs. 98 (`F_FULLFSYNC`); group-commit speedup 4.4× vs. 4.3×—so over this 2× dimension change the tax is dominated by the device flush, not the graph work. We caution that this is two points, not a law: at much higher dimension (e.g. GIST's 960-d, where per-insert graph work grows while the flush cost is fixed) we would expect the *ratio* to shrink, and confirming that is future work. We also re-ran the tax on *real* SIFT1M embeddings rather than synthetic vectors, and the numbers are consistent to within ∼10% (`fsync` floor 48,810/322; per-write 408/111 inserts/s, the latter's own 7-trial range 105–119; group-commit 4.0×), confirming the tax is a property of the durable-write path, not of the vector distribution.

5 Group Commit for a Graph-ANN WAL

Group commit is a classic database technique [16]: batch several log records and issue a single `fsync` for the group, decoupling per-operation commit latency from flush latency. We add a deferred-sync mode to the mutable segment: during a batch, WAL appends skip the per-record `fsync`, and a single `commitDeferredSync()` at the end `fsync`s once for the whole batch. The public batch-insert path uses it; single inserts keep per-write durability.

The sweep (Table 3, Fig. 1) makes the earlier reviewers' concern concrete: the group-commit speedup is not a single constant but a curve in batch size, and its magnitude depends on the `fsync` mode. Under honest durability it climbs from 1× (batch 1, 102/s) to 4.3× (batch 2,000, 433/s) by paying for one flush rather than N; under plain `fsync` it stays near 1× because the flush was never the bottleneck. Note that the group-committed `F_FULLFSYNC` rate (433/s) essentially matches the per-write *plain*-`fsync` rate (442/s, Table 2): once the flush leaves the per-insert critical path, HNSW graph insertion is again the bound in both modes—which is also why the two curves in Fig. 1 meet. (The batch-1 column here and the per-write row of Table 2 are independent 7-trial runs of the nominally identical single-insert operating point and agree closely (102 vs. 98/s `F_FULLFSYNC`, 481 vs. 442/s plain); the small (∼9%) plain-mode offset reflects `batchInsert`'s lower per-call overhead, and reinforces that the ranges, not the point estimates, carry the signal.)

Table 3: Group-commit batch-size sweep (Apple M4 Pro, d=128, N=2,000 inserts/trial, one `fsync` per batch), inserts/s as median of 7 trials (min–max), and speedup over batch 1. Under `F_FULLFSYNC` the speedup climbs a curve to 4.3×; under plain `fsync` batching barely helps because the flush was already page-cache-cheap. Speedups are computed on unrounded medians.

	`F_FULLFSYNC` (honest)		Plain `fsync`	
Batch	Inserts/s	Speedup	Inserts/s	Speedup
1	102 (97–109)	1.0×	481 (415–491)	1.0×
10	339 (318–356)	3.3×	498 (453–533)	1.0×
50	402 (391–443)	4.0×	509 (462–560)	1.1×
200	419 (410–456)	4.1×	496 (443–563)	1.0×
1000	431 (411–461)	4.2×	514 (440–539)	1.1×
2000	433 (418–461)	4.3×	507 (447–543)	1.1×

Crash consistency. Group commit is only correct if the batch's single `fsync` makes the *whole* batch durable before the call returns. We validate this with a fault-injection test: a child process performs a group-committed batch of N inserts and then calls `_exit(0)` *without* a clean shutdown (simulating a crash immediately after commit); the parent reopens the store and asserts all N rows recover. It passes. The WAL is protected by a CRC-32 over the header and payload (an earlier version used a payload-only FNV-1a mislabeled as "crc32"—replaced here), and replay truncates at the first bad record. Because the child exits on a live OS, this validates crash-consistency (no acknowledged write lost, no torn record) but not power loss, which requires block-level injection (§10).

6 Recall-Bounded Durability

Group commit trades durability for throughput by leaving a window of W acknowledged-but-not-yet-`fsync`'d inserts; a crash loses those W most-recent vectors. Because this is an ANN index, we can express that exposure in the unit the application actually cares about—*recall*—and bound it. Define the *recall at risk* $R(W)$ as the fraction of a query's top-k results that lie in the un-durable window and would therefore vanish on a crash.

Two bounds frame $R(W)$. **(i) Distribution-free worst case.** A query's top-k are k distinct vectors, so at most $\min(W, k)$ of them can be un-durable: $R(W) \le \min(W, k)/k$ for *any* workload. To *guarantee* "a crash never costs more than ε recall@k" one sets $W \le \varepsilon k$—but that forces W to a handful of records, essentially per-write `fsync`. **(ii) Benign expectation.** If insertion order is independent of the query distribution, the W un-durable inserts are a uniform random subset of the N indexed vectors, so $\mathbb{E}[R(W)] = W/N$—linear in the window and independent of k.

Between these two bounds lies what actually happens, and it depends on how correlated inserts are with queries. Fig. 2 measures $R(W)$ on SIFT1M (N=10^6, k=10) in two regimes. *Benign* (insertion order uncorrelated with queries): the empirical curve follows W/N (sitting modestly below it—0.13% measured vs. 0.20% expected at W=2,000) and stays ∼770× below the worst case—so at our group-commit batch (W=2,000) a crash risks only 0.13% of recall@10, versus the 100% the distribution-free bound admits, while delivering the 4.3× throughput of Table 3. *Adversarial* (the un-durable window is exactly the query-hot vectors—i.e. "query the freshly-inserted data"): R rises steeply, 21% at W=2,000 and 100% by W=10^4, so here small windows are mandatory. A real deployment sits between these, set by its insert/query correlation.

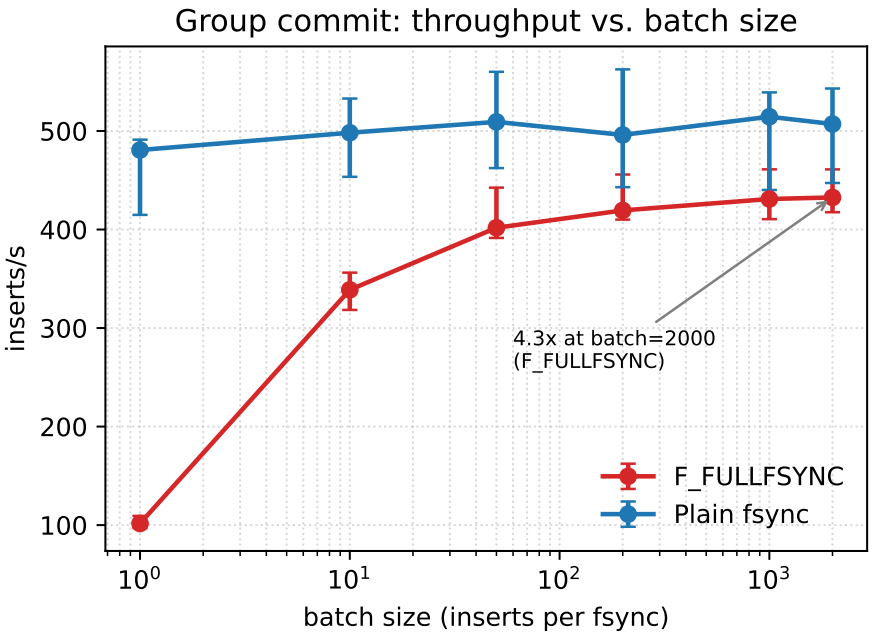

Figure 1: Group commit throughput vs. batch size (Apple M4 Pro, d=128; median of 7 trials, min–max whiskers). Under `F_FULLFSYNC` the curve rises from 102 to 433 inserts/s (4.3×) as the flush is amortized; under plain `fsync` it is flat, because the page-cache "flush" costs almost nothing. The two converge at large batches: once the flush leaves the critical path, HNSW graph insertion is the bound.

This yields a *recall-aware commit* policy: pick the largest batch W whose $R(W) \leq \varepsilon$ for the deployment's regime—$W{=}\varepsilon N$ under the benign expectation (enormous: ε=1% allows $W{=}10^4$ on SIFT1M, i.e. full group-commit throughput), tightening toward $W{=}\varepsilon k$ as inserts become query-correlated (the adversarial curve shows ε=1% then needs $W \approx 10^2$). To our knowledge this is the first durability guarantee stated in recall units for an ANN index; a crash-time enforcement implementation and a proof under weaker-than-independence assumptions are future work. The construction is ANN-specific: it exploits that an approximate index already tolerates small recall perturbations, so a bounded loss of the newest vectors is a principled, quantifiable relaxation rather than a correctness bug.

7 Fast, Crash-Consistent Recovery

Recovery time is a first-class operational metric that ANN work has rarely reported for a standalone index; the closest is P-HNSW [11], which compares WAL-replay vs. snapshot-load recovery for HNSW on *persistent memory*. We report the analogous comparison for conventional `fsync`/WAL-to-disk storage. Our store has two recovery paths: a *sealed* segment loads its HNSW snapshot directly (no graph rebuild), while a *mutable* segment replays its WAL, re-inserting each record through the graph builder. Table 4 shows snapshot load grows far more slowly than WAL replay: over N=1k–6k the sealed path rises only ∼20% (39 to 47 ms) while replay grows ∼7.5× (with mildly superlinear per-insert cost as the graph grows), so the gap widens from ∼5× to ∼32×. At much larger N snapshot load is itself linear in the serialized size—P-HNSW [11] reports this at 1–10M—so the durable statement is a far smaller per-element constant, not a constant recovery

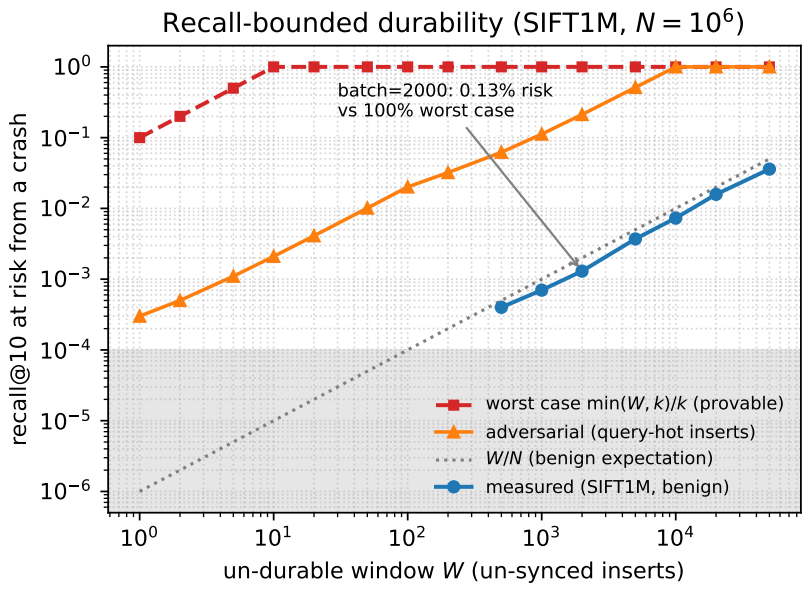

Figure 2: Recall-bounded durability on SIFT1M (N=10^6). Recall@10 at risk from a crash, $R(W)$, vs. the un-durable window W: the distribution-free worst case $\min(W, k)/k$ (provable), the benign expectation W/N, the measured benign value, and the measured *adversarial* value (un-durable window = query-hot vectors). The benign exposure sits ∼770× below the worst case (0.13% at W=2,000, our group-commit batch); the adversarial curve rises to 100% by W=10^4, so the safe window depends on insert/query correlation. Shaded band = measurement resolution (10^{-4}).

time. We read the *shape* (constant vs. growing) as the result, and note two caveats on the absolute values. First, the comparison is deliberately apples-to-oranges: it charges WAL replay for rebuilding the graph but does *not* charge the sealed path for the snapshot-creation (sealing) work that must have happened earlier to produce the snapshot—that sealing cost is unmeasured here. The table therefore shows that *reading* a snapshot is cheap, not that snapshotting is free. Second, it is one host and one filesystem. (With 7 trials the sealed column is now monotonic in N, 38.9 → 44.2 → 47.0 ms, confirming the earlier single-run non-monotonicity was jitter around a slowly-growing cost.)

The practical consequence: because WAL replay pays a full graph rebuild while snapshot load pays a much smaller per-element cost, an unsealed WAL that grows large eventually exceeds any recovery-time budget—the operational justification for periodic sealing. Extending the sweep confirms the trend holds an order of magnitude further: the gap grows monotonically to ∼157× at N=20k (sealed ∼37 ms vs. WAL replay ∼5.9 s), replay staying $O(N)$ while snapshot load grows far more slowly. (Recovery is `fsync`-mode-independent for the replay path, so this extension was run under plain `fsync`; the sealed path differs only by a small constant.) Charting the budget crossover on real media, and validating consistency under true power loss (Linux `dm-log-writes`, harness in §10), remain the main hardware-gated measurements.

8 Index Competitiveness

So that the durability results are not measured on a broken index, we confirm the HNSW index is competitive on the standard SIFT1M benchmark (1M 128-d vectors, exact 100-NN ground truth shipped with the dataset). Table 5 traces recall@10 vs. single-thread QPS; recall is ID-set recall

Table 4: Cold-open recovery time, sealed (snapshot load) vs. mutable (WAL replay), Apple M4 Pro, d=128, `F_FULLFSYNC`; median of 7 trials (min–max). Snapshot-creation/sealing cost is *not* included—only the cost of reading an already-sealed snapshot. WAL-replay time is dominated by graph rebuild and is `fsync`-mode-independent.

N	Sealed (ms)	WAL replay (ms)	Gap
1,000	38.9 (32.4–46.8)	195.5 (181.7–196.5)	5.0×
3,000	44.2 (33.8–55.0)	650.7 (647.0–683.9)	14.7×
6,000	47.0 (42.7–48.4)	1,483.6 (1,455.6–1,602.0)	31.6×

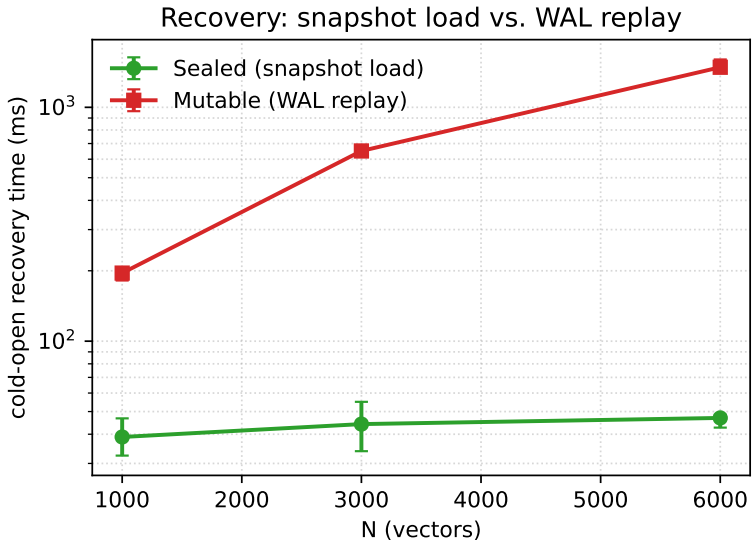

Figure 3: Cold-open recovery vs. N (Apple M4 Pro, d=128; median of 7 trials, min–max whiskers, log y). Snapshot load is near-flat; WAL replay grows ∼linearly.

against the shipped ground truth. Recall climbs monotonically with ef from 0.77 to 0.999—the shape and absolute levels expected of a correct HNSW—reaching 0.977 at ∼5.9k QPS (ef=64) and 0.997 at ∼2.2k QPS (ef=200). The full graph builds in 504 s single-threaded with a peak index arena of 999 MiB. As a head-to-head baseline we ran hnswlib [2]—the reference implementation—on the identical dataset, parameters, and host (Table 5). Both indexes use identical M and $\text{ef}_{\text{construction}}$, single-threaded queries, the same recall@10 harness against the shipped ground truth, and the same host. At every ef our recall is deterministically a little higher (e.g. at ef=100, 0.990 vs. 0.983) at QPS within a couple of percent (single-run, so within timing noise; at ef=500 hnswlib is nominally faster, 1,040 vs. 1,023). We therefore read the two as Pareto-equivalent (Fig. 4); the one cost we pay is build time (504 s vs. hnswlib's 394 s, ∼1.3×). The index is thus genuinely competitive, not merely functional, before we layer durability on top. Hot-path optimization (a versioned visited set, FMA distance kernels, devirtualized dispatch, and a pooled graph arena) was validated separately on 30k clustered 128-d vectors, where recall reaches 1.0 and single-thread QPS reaches ∼97k at ef=10; the pooled arena cut peak index memory on that workload from 2 GiB to 38 MiB (∼54×).

GIST1M (960-d). On the higher-dimensional GIST1M benchmark (1M 960-d vectors, shipped exact 100-NN ground truth) the same picture holds against hnswlib, more strongly. Recall@10

Table 5: Recall@10 vs. single-thread QPS on SIFT1M (Apple M4 Pro, d=128, n=10^6, M=16, $\text{ef}_{\text{construction}}$=200), ours vs. hnswlib on the identical dataset/params/host, measured against the dataset's shipped exact ground truth (ID-set recall@10). Recall is deterministic at fixed parameters; QPS is a single-run timing (unlike the median-of-7 durability numbers). Build: ours 504 s (peak arena 999 MiB), hnswlib 394 s.

	this work		hnswlib	
ef	recall@10	QPS	recall@10	QPS
10	0.770	22,190	0.709	20,829
32	0.931	10,114	0.904	9,907
64	0.977	5,851	0.963	5,758
100	0.990	4,045	0.983	3,903
200	0.997	2,240	0.996	2,200
500	0.999	1,023	0.999	1,040

Table 6: Recall@10 vs. single-thread QPS on GIST1M (Apple M4 Pro, d=960, n=10^6, M=16, $\text{ef}_{\text{construction}}$=200), ours vs. hnswlib on the identical dataset/params/host, ID-set recall@10 against the shipped exact ground truth. Build: ours 2,567 s. QPS is single-run.

	this work		hnswlib	
ef	recall@10	QPS	recall@10	QPS
10	0.484	5,381	0.409	4,038
32	0.709	2,417	0.635	1,977
64	0.825	1,400	0.756	1,198
100	0.879	963	0.826	839
200	0.940	526	0.907	475
500	0.984	238	0.964	220

climbs monotonically from 0.48 at ef=10 to 0.984 at ef=500 (Table 6, Fig. 5), and at every ef our index is above hnswlib on *both* axes—averaging +5.5 recall points at 1.18× QPS (e.g. 0.940/526 QPS vs. 0.907/475 QPS at ef=200). The full graph builds in 2,567 s (peak graph arena 999 MiB—the same as SIFT1M because the pooled arena holds adjacency lists only, not the vectors, so it is dimension-independent). GIST's weaker neighbor structure and 7.5× higher per-distance cost give lower recall-per-ef and ∼4× lower QPS at matched ef than SIFT1M for both indexes (and a larger gap, ∼15–25×, at matched recall), consistent with published HNSW behavior. We are deliberately cautious about the margin: it is larger than on SIFT1M and rests on single-run QPS, so although both indexes use matched M and $\text{ef}_{\text{construction}}$, a controlled ef-semantics/tie-handling parity study is left to the full version. We therefore read GIST1M as "at least on par with hnswlib," plausibly better on high-dimensional data where diversifying neighbor selection helps most—the point being, again, that the durability results sit on a competitive index.

9 Related Work

Graph ANN and updates. HNSW [1] and DiskANN [3] are the dominant graph indexes; FreshDiskANN [4] and SPFresh [5] add streaming updates (out-of-place merge and in-place

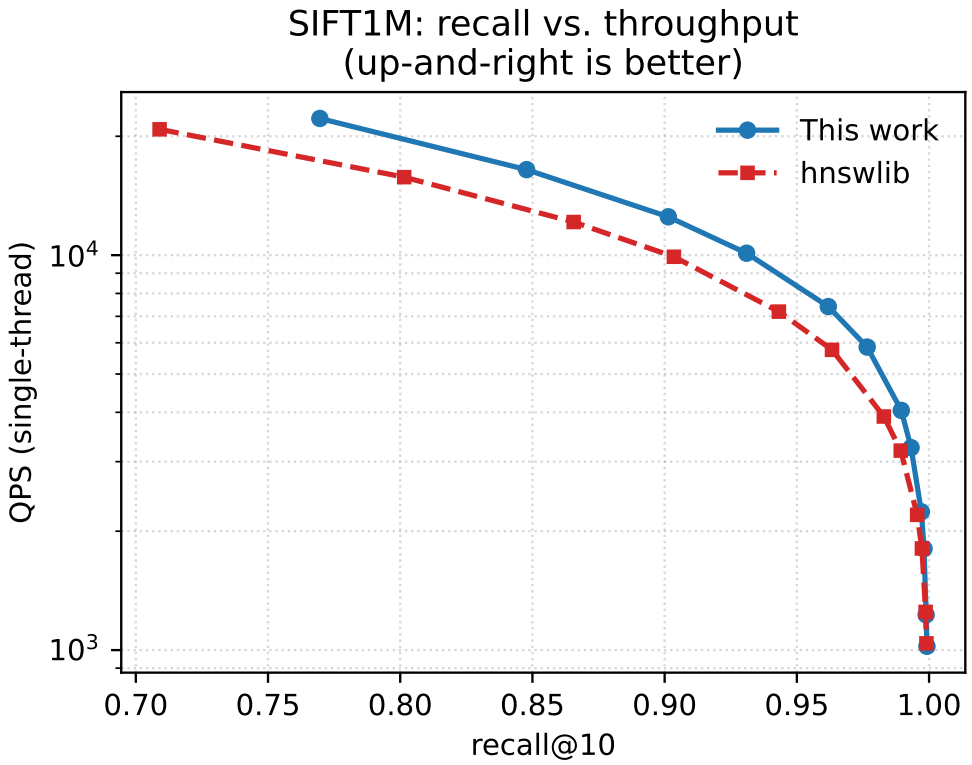

Figure 4: SIFT1M recall–QPS Pareto front (single-thread, Apple M4 Pro). Our index (solid) tracks hnswlib (dashed) across the operating range: over matched ef it averages +1.8 recall points (deterministic) at a mean 1.02× QPS (single-run, within timing noise), i.e. the two are Pareto-equivalent. Up-and-to-the-right is better.

rebalancing, respectively), and each describes a redo-log/snapshot crash-recovery mechanism (FreshDiskANN even reports coarse recovery times of minutes at 5–30M points; SPFresh pairs a snapshot with a WAL), while LSM-VEC [8] distributes a proximity graph across LSM levels. None, however, isolate and *quantify* an `fsync`/durability tax against a non-durable baseline, and none report a controlled snapshot-load-vs-WAL-replay recovery comparison for a standalone block-storage ANN; LSM-VEC in particular is silent on `fsync`/recovery despite an LSM design. **Durability for graph ANN.** Closest to our recovery analysis is P-HNSW [11], which makes a standalone HNSW crash-consistent on *persistent memory* and compares WAL-replay vs. snapshot-load recovery as a function of index size. We differ in substrate and in the headline metric: we target conventional block storage, where durability is an `fsync`/WAL-to-disk cost carrying a platform trap (plain `fsync` vs. `F_FULLFSYNC` drive-cache flush) that has no analogue in the PMEM persist-barrier model, and we isolate a portable `fsync`-mode tax—durable vs. non-durable on the same index—that persistent memory does not expose. **Disk-resident ANN.** Starling [6] optimizes on-disk graph layout *statically*; SPANN [7] splits centroids/posting lists across RAM/SSD. **Durability inherited, not analyzed.** Azure Cosmos DB embeds DiskANN and inherits durability/recovery from the host engine [9], so it does not isolate an ANN-specific recovery cost; production vector DBs (Milvus [12], Qdrant, Weaviate) ship WAL persistence but publish no such analysis. **Benchmarks.** ANN-Benchmarks [13], Big ANN [14] (including its in-memory streaming track), and VectorDBBench [15] measure recall/QPS/cost but have no

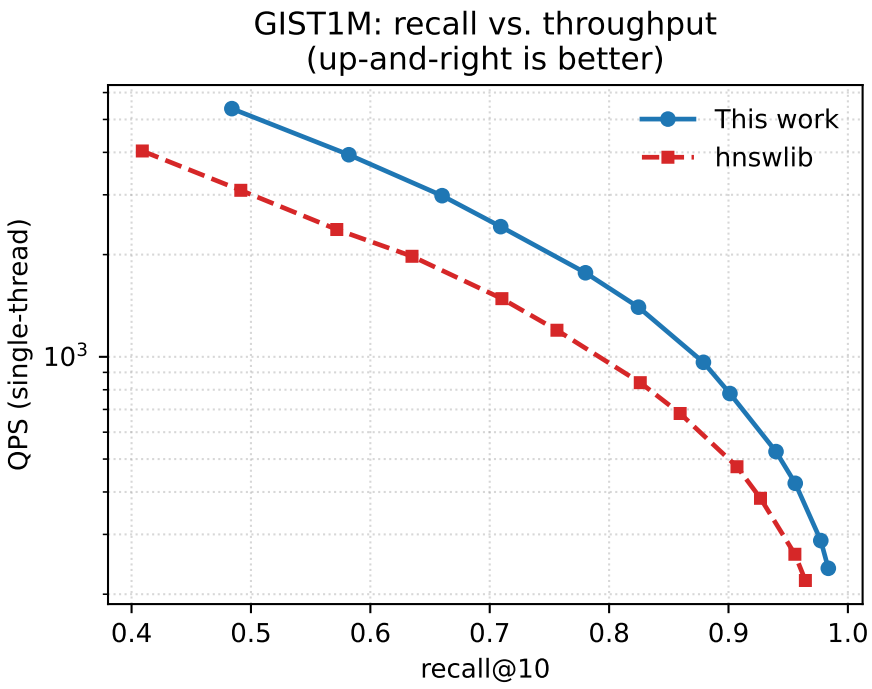

Figure 5: GIST1M recall–QPS Pareto front (single-thread, Apple M4 Pro). Our index (solid) is above hnswlib (dashed) on both axes across the range (+5.5 recall points, 1.18× QPS on average at matched ef); QPS is single-run.

crash-recovery or `fsync` case. **Durability techniques.** Group commit dates to early transaction systems and was refined in Aether [16]; we apply it to a graph-ANN WAL and characterize its ANN-specific effect.

10 Threats to Validity and Remaining Work

Numbers here are medians of 7 trials on a single host (Apple M4 Pro, APFS/NVMe) at the SIFT dimensionality (d=128); every durability number is labeled by `fsync` mode and reported with its min–max range. The primary tables use synthetic vectors, but we verified the tax reproduces on real SIFT1M embeddings within the trial spread (§4), so the vector distribution is not a confound. Because macOS `fsync` does not flush the drive cache, the honest figures use `F_FULLFSYNC`, and the ∼155× floor gap is a macOS/APFS artifact that will be smaller on Linux/NVMe. A Linux host with `ext4`/`btrfs` on NVMe (and `dm-log-writes` for true power-loss injection) is needed to generalize. Remaining work before a full version: (i) recovery on real media and the sealing-budget crossover, including the unmeasured snapshot-creation cost; (ii) a recovery/tax N-sweep up to the 10^6 scale of the competitiveness study; (iii) a concurrent (leader/follower) group committer; (iv) crash-time enforcement of the recall-bounded commit policy (§6) and a proof under weaker-than-independence assumptions; and (v) multi-host generalization with true power-loss injection. The single-host and macOS-artifact limitations are the principal threats to external validity.

11 Conclusion

Durability is the axis ANN evaluation forgot. This paper's contribution is a way to measure it honestly rather than any new storage mechanism: a protocol (tmpfs guard, `fsync`-floor, `fsync`-mode labeling) plus an illustrative single-host characterization built with it. Applied to a competitive HNSW index (on the hnswlib recall–QPS front), the protocol shows durability is measurable, that honest `fsync`-mode labeling can change a reported number by orders of magnitude (the macOS/APFS `fsync` trap), that textbook group commit offsets the tax along a batch-size curve (up to 4.3× under honest durability) while staying crash-consistent under a fork-and-crash test, and that reading a snapshot is far cheaper than WAL replay. Beyond measurement, we introduce one ANN-native mechanism: *recall-bounded durability*, which expresses a crash's damage in recall units and shows the real exposure of group commit is ∼770× below the distribution-free worst case—so a recall-staleness SLA and full throughput coexist. These figures are medians of 7 trials on one host, not general claims; the transferable results are the measurement discipline and the recall-bounded framing. We hope the honest-measurement protocol becomes a default for durable ANN systems, and that a full study (multiple hosts, real media, and power-loss injection) puts numbers on it.

Reproducibility

All code, benchmarks, and tests are open. Key entry points: `make bench-durability` reproduces Tables 1–4 and Figs. 1,3 (default d=128, 7 trials, plain `fsync`; one `fsync` mode per invocation, so add `--full-fsync` for honest durability, or run `run_durability_all.sh` for both modes at d=64, 128). `make bench-ann` and `make bench-hnswlib` (with `--data datasets/sift`) reproduce Table 5 and Fig. 4. `make test` runs the functional suite; the fuzz harnesses (`make fuzz`) build under AddressSanitizer and UndefinedBehaviorSanitizer, and the concurrency tests additionally run clean under ThreadSanitizer. Figures are regenerated from the emitted CSVs by the scripts under `scripts/`.

References

[1] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. doi:10.1109/TPAMI.2018.2889473. arXiv:1603.09320.

[2] Yu. A. Malkov and D. A. Yashunin. hnswlib: A header-only C++/Python library for fast approximate nearest neighbor search. GitHub repository, `nmslib/hnswlib`, Apache-2.0, 2016–present. `https://github.com/nmslib/hnswlib`.

[3] S. J. Subramanya, Devvrit, H. V. Simhadri, R. Krishnaswamy, and R. Kadekodi. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 13748–13758, 2019.

[4] A. Singh, S. J. Subramanya, R. Krishnaswamy, and H. V. Simhadri. FreshDiskANN: A fast and accurate graph-based ANN index for streaming similarity search. arXiv preprint arXiv:2105.09613, 2021.

[5] Y. Xu, H. Liang, J. Li, S. Xu, Q. Chen, Q. Zhang, C. Li, Z. Yang, F. Yang, Y. Yang, P. Cheng, and M. Yang. SPFresh: Incremental in-place update for billion-scale vector search. In *Proc. 29th ACM Symposium on Operating Systems Principles (SOSP)*, pages 545–561, 2023. doi:10.1145/3600006.3613166.

[6] M. Wang, W. Xu, X. Yi, S. Wu, Z. Peng, X. Ke, Y. Gao, X. Xu, R. Guo, and C. Xie. Starling: An I/O-efficient disk-resident graph index framework for high-dimensional vector similarity search on data segment. *Proc. ACM Manag. Data (SIGMOD)*, 2(1):Article 14, 2024. doi:10.1145/3639269.

[7] Q. Chen, B. Zhao, H. Wang, M. Li, C. Liu, Z. Li, M. Yang, and J. Wang. SPANN: Highly-efficient billion-scale approximate nearest neighbor search. In *Advances in Neural Information Processing Systems 34 (NeurIPS)*, 2021. arXiv:2111.08566.

[8] S. Zhong, D. Mo, and S. Luo. LSM-VEC: A large-scale disk-based system for dynamic vector search. arXiv preprint arXiv:2505.17152, 2025.

[9] N. Upreti, H. V. Simhadri, H. S. Sundar, K. Sundaram, et al. Cost-effective, low-latency vector search with Azure Cosmos DB. *Proc. VLDB Endowment (PVLDB)*, 18(12):5166–5183, 2025. arXiv:2505.05885.

[10] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou. The Faiss library. arXiv preprint arXiv:2401.08281, 2024.

[11] H. Lee, T. Park, Y. Na, and W.-H. Kim. P-HNSW: Crash-consistent HNSW for vector databases on persistent memory. *Applied Sciences*, 15(19):10554, 2025. doi:10.3390/app151910554.

[12] J. Wang, X. Yi, R. Guo, H. Jin, P. Xu, S. Li, X. Wang, X. Guo, C. Li, X. Xu, K. Yu, Y. Yuan, Y. Zou, J. Long, Y. Cai, Z. Li, Z. Zhang, Y. Mo, J. Gu, R. Jiang, Y. Wei, and C. Xie. Milvus: A purpose-built vector data management system. In *Proc. 2021 ACM SIGMOD International Conference on Management of Data*, pages 2614–2627, 2021. doi:10.1145/3448016.3457550.

[13] M. Aumüller, E. Bernhardsson, and A. Faithfull. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems*, 87:101374, 2020. doi:10.1016/j.is.2019.02.006.

[14] H. V. Simhadri, M. Aumüller, A. Ingber, M. Douze, et al. Results of the Big ANN: NeurIPS'23 competition. arXiv preprint arXiv:2409.17424, 2024.

[15] Zilliz. VectorDBBench: A benchmark tool for vector databases. Open-source software, GitHub, 2023. `https://github.com/zilliztech/VectorDBBench`.

[16] R. Johnson, I. Pandis, R. Stoica, M. Athanassoulis, and A. Ailamaki. Aether: A scalable approach to logging. *Proc. VLDB Endowment (PVLDB)*, 3(1):681–692, 2010. doi:10.14778/1920841.1920928.

[17] T. S. Pillai, V. Chidambaram, R. Alagappan, S. Al-Kiswany, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau. All file systems are not created equal: On the complexity of crafting crash-consistent applications. In *Proc. 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 433–448, 2014.

[18] M. Zheng, J. Tucek, D. Huang, F. Qin, M. Lillibridge, E. S. Yang, B. W. Zhao, and S. Singh. Torturing databases for fun and profit. In *Proc. 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 449–464, 2014.

[19] J. Corbet. PostgreSQL's `fsync()` surprise. LWN.net, 2018. `https://lwn.net/Articles/752063/`.